DL 201 · 3 CREDITS · 13 WEEKS · FOUNDATION COURSE

Introduction to Deep Learning

Foundations of Neural Networks with PyTorch

TERM Fall 2026	**FORMAT** 3 h lecture + 2 h practice/week	**CREDITS** 3
LECTURE *Tue / Thu, time TBA*	**PRACTICE** *Fri, time TBA*	**LOCATION** *Room TBA*
INSTRUCTOR *Name TBA*	**OFFICE HOURS** *TBA*	**CONTACT** *email TBA*

PREREQUISITE **Introduction to Machine Learning**	**THIS COURSE** **Introduction to Deep Learning**	**FEEDS INTO** **Advanced LLMs and Computer Vision**

Fields marked TBA are placeholders to fill in with the section's specifics (instructor, meeting days and times, room, and contact).

1 Course Description

This is the foundational deep-learning course, required of every computer-science student and taught across all specializations. It assumes students have completed an introductory machine-learning course and turns that background into working neural-network skills: framing a task in tensor terms, and then implementing, training, and debugging networks in PyTorch. The emphasis throughout is on building and experimentation, not on watching.

The architectures, training discipline, and representation-learning ideas built here form the shared deep-learning base for continued specialization in artificial intelligence, including **large language models** and **computer vision**. The course is valuable in its own right, not only as a step toward later courses: the goal is a solid, transferable foundation for any AI-related path.

The course is project- and lab-based. Every week pairs a lecture and a practice lesson with a hands-on lab students build themselves, and the term ends in a project students take from framing to a working, evaluated model. The labs are designed for the way students will actually work, with an AI coding assistant at hand. Section 5 explains how that works.

2 Learning Outcomes

On successful completion of this course, students will be able to:

1. **Frame tasks as networks.** Translate ML tasks into neural-network terms: tensor inputs, model outputs, and loss functions.
2. **Represent data as tensors.** Use tensor operations, shapes, broadcasting, and device handling fluently.
3. **Build data pipelines.** Construct Dataset and DataLoader pipelines for batching, shuffling, and transforms.

4. **Apply autodiff.** Use automatic differentiation (autograd) to compute gradients that drive learning.
5. **Implement networks in PyTorch.** Turn an architecture specification into working, trainable modules.
6. **Reason about optimization.** Understand the dynamics of SGD and Adam, learning rates, and convergence.
7. **Diagnose training issues.** Detect and fix overfitting, underfitting, and unstable or stalled training.
8. **Build core architectures.** Implement MLPs, CNNs, RNNs, LSTMs/GRUs, and autoencoders.
9. **Apply representation learning.** Learn useful features with autoencoders and contrastive methods.
10. **Stand on a transferable base.** Carry these foundations into advanced study of language models and computer vision.

3 Prerequisites

- **Required: Introduction to Machine Learning** (or equivalent). Students should be comfortable with classification and regression, train/validation/test splits, overfitting and generalization, and basic model evaluation.
- **Programming:** proficiency in Python (functions, classes, NumPy-style array thinking).
- **Mathematics:** linear algebra (vectors, matrices, matrix multiplication) and multivariable calculus (partial derivatives, gradients, the chain rule).
- **Probability:** basic probability and statistics (distributions, expectation, mean, and variance).

4 Course Format

Each week has three parts. A **3-hour lecture** develops the concepts and theory. A **2-hour practice lesson** follows, in which the instructor demonstrates implementations, runs code, and works through examples. A weekly lab is then set as homework, where students implement, experiment, read the curves, and explain what they found. The labs are the graded hands-on core of the course.

The thirteen weeks are organized into four parts: **Foundations** (weeks 1 to 3), **Training Infrastructure** (weeks 4 to 7), **Architectures and Representation Learning** (weeks 8 to 11), and **Integration** (week 12). A mid-term mini-project consolidates Parts I and II; a final project consolidates the entire course and points toward the advanced electives.

5 Working with an AI assistant: How the Labs Are Built

Students are expected to use an AI assistant as a coding partner in this course. That is how practitioners work now, and pretending otherwise would not prepare them for the advanced

courses or for practice. The labs are designed around that reality: An AI assistant can write the boilerplate, but it cannot do the learning for them. Every weekly lab, therefore, has three parts.

PART A · AI ASSISTANT WELCOME	PART B · STUDENT REASONING	PART C · STUDENT'S WORDS
Build	**Predict & probe**	**Explain & defend**
Students produce working code that meets a specification or reaches a target metric, using an AI assistant freely for scaffolding, syntax, and debugging.	Before running anything, students write down what they expect (a shape, a curve, which setting wins). They then run controlled experiments and compare. The gap between prediction and result is the learning.	Students submit a short write-up or annotated code explaining why the solution works, where it would break, and what they changed and why. They must be able to defend any line they submit.

What this means in practice

- Grading weight sits on **Predict** and **Explain**, the parts where copying an answer does not help them.
- Several labs ask students to **critique an AI-generated solution** that contains a subtle, realistic mistake (a wrong loss, a data leak, a misplaced normalization). Finding it requires understanding, not generation.
- Labs are framed as **predict, build, break, and explain**, so an answer a student cannot reason about is worth little.
- The final project includes a short **oral defense**, so what is assessed is the student's understanding, not who or what typed the code.
- **Disclosure.** Students note where they used the AI assistant. Using it is encouraged; submitting work they cannot explain is an integrity violation (Section 9).

6 Weekly Schedule

WK	LECTURE TOPIC (3 H)	WEEKLY LAB (HOMEWORK): BUILD WITH AN AI ASSISTANT, THEN LEARN IT
PART I · FOUNDATIONS (WEEKS 1 TO 3)		
1	**Deep Learning Overview & ML-to-Network Framing** From the introductory ML course to networks, tasks as tensors in, loss out, tooling, and environment.	**Build:** a tiny end-to-end training loop with AI-assistant scaffolding. **Learn it:** for three ML tasks, write the input tensor shape, output layer, and loss in plain language, and predict each before checking.
2	**Tensors & Data Representation** Tensor operations, shapes, broadcasting, devices; images, text, and tabular data as tensors.	**Build:** a set of tensor ops with an AI assistant's help. **Learn it:** predict the output shape of eight broadcasting and reshape expressions before running, then fix a seeded shape-mismatch bug.
3	**MLPs & Backpropagation** Multilayer perceptrons; the forward pass; backprop mechanics via PyTorch autograd.	**Build:** an MLP trained with autograd. **Learn it:** hand-derive the gradient for one layer, check it against autograd, and explain

WK	LECTURE TOPIC (3 H)	WEEKLY LAB (HOMEWORK): BUILD WITH AN AI ASSISTANT, THEN LEARN IT
		in words what each gradient tells the weight to do.
PART II · TRAINING INFRASTRUCTURE (WEEKS 4 TO 7)		
4	**Data Pipelines** The Dataset and DataLoader abstractions; batching, shuffling, transforms, and splits.	**Build:** a custom Dataset and full DataLoader. **Learn it:** introduce a deliberate data leak, observe the inflated metric, explain it, and predict the effect of batch size on the curve.
5	**Loss Functions & Metrics** Task-appropriate losses (cross-entropy, MSE, BCE); metrics; the train/eval loop.	**Build:** a training loop with metric tracking. **Learn it:** pick the wrong loss on purpose, predict and observe the failure, then critique an AI-written accuracy metric that is wrong under class imbalance.
6	**Optimization** Gradient descent; SGD, momentum, and Adam; learning rates and optimization dynamics.	**Build:** an optimizer-comparison harness. **Learn it:** predict the loss-curve shape for three learning rates before running, explain divergence and slow convergence by step size, then tune to a target accuracy.
7	**Regularization & Generalization** Overfitting, dropout, weight decay, early stopping, and basic data augmentation.	**Build:** add dropout and weight decay. **Learn it:** first make a model overfit, then close the train/validation gap to a target and explain which regularizer helped and why.
PART III · ARCHITECTURES & REPRESENTATION LEARNING (WEEKS 8 TO 12)		
8	**Convolutional Networks I** Convolution, pooling, and feature maps; building a CNN image classifier.	**Build:** a CNN image classifier. **Learn it:** compute by hand the output spatial size and parameter count of each conv and pool layer, then verify against the model summary.
9	**Convolutional Networks II** Batch and layer normalization; residual connections; modern CNN design.	**Build:** add normalization and residual blocks. **Learn it:** ablate each, predict and measure the effect on trainability and depth, and explain why residuals help gradient flow. *(Mid-term mini-project starts.)*
10	**Recurrent Networks (RNNs)** Sequence data and recurrence; the RNN cell; backpropagation through time; the vanishing and exploding gradient problem.	**Build:** a plain RNN on a character-level or short time-series task. **Learn it:** measure gradient norms across time steps, demonstrate vanishing gradients on long sequences, and explain why long-range dependencies are hard for a plain RNN.
11	**LSTMs, GRUs & Sequence Tasks** Gated recurrent units; how gates restore gradient flow; LSTM versus GRU; sequence classification and sequence-to-sequence tasks.	**Build:** an LSTM or GRU on the same task and compare it to the plain RNN. **Learn it:** ablate the gates, predict behavior on long versus short sequences, and explain how

WK	LECTURE TOPIC (3 H)	WEEKLY LAB (HOMEWORK): BUILD WITH AN AI ASSISTANT, THEN LEARN IT
		gating preserves the gradient signal where the RNN failed.
12	**Representation Learning** Autoencoders and latent representations; contrastive methods for learned features.	**Build:** an autoencoder and a contrastive embedding. **Learn it:** probe the latent space (interpolate, cluster), interpret what it captures, and critique an AI-suggested but flawed contrastive loss.
PART IV · INTEGRATION (WEEK 13)		
13	**Integration & Transfer Learning** Transfer learning and fine-tuning; model inference; the end-to-end workflow in the advanced courses.	**Build:** fine-tune a pretrained model end-to-end. **Learn it:** compare from-scratch, fine-tuning, and frozen-features under a fixed budget, and explain when transfer helps. *(Final project due.)*

The schedule expands the four-part course outline into per-week detail. Topics may shift by a session to match class pace.

7 Assessment & Grading

Grading is project- and lab-based, with weight placed on the parts that an AI assistant cannot do for the student: reasoning, interpretation, and defense. There are no written exams.

COMPONENT	WHAT IT COVERS	WEIGHT
Weekly labs	Eleven labs (best 10 count), each graded mostly on the Predict and Explain parts, not on the code alone.	**40%**
Mid-term mini-project	A CNN-based project (around weeks 9 to 10): build, train, ablate, and report on an image task.	**20%**
Final project	Implementation, a written report with results, an AI-use reflection, and a short oral defense.	**35%**
Participation	In-class exercises and participation.	**5%**
Total		**100%**

Final project

Working solo or in a pair, students take a task end to end: frame it, build a suitable architecture from the families covered in the course, train and tune it, diagnose failures, and report results. Students may use an AI assistant throughout. Deliverables are a one-page proposal (around week 8), a code submission, a short report with curves and discussion, a half-page reflection on what the AI assistant did versus what they had to work out themselves, and a short oral defense in which they explain their design decisions. The oral defense is where understanding, rather than authorship, is confirmed.

8 Tools & Resources

- **Core stack:** Python 3.11+, PyTorch, NumPy, and Matplotlib; Jupyter notebooks or Google Colab for the labs.
- **AI assistant:** An AI assistant is part of the toolkit and is expected for the Build portion of the labs.
- **Compute:** a GPU is helpful but not required; Colab provides free GPU access for the labs and project.
- **Reference reading:** Dive into Deep Learning (d2l.ai) and the official PyTorch tutorials, used as companions to lecture.
- **Course materials:** lecture slides, lab notebooks, and starter code are posted weekly on the course site.